

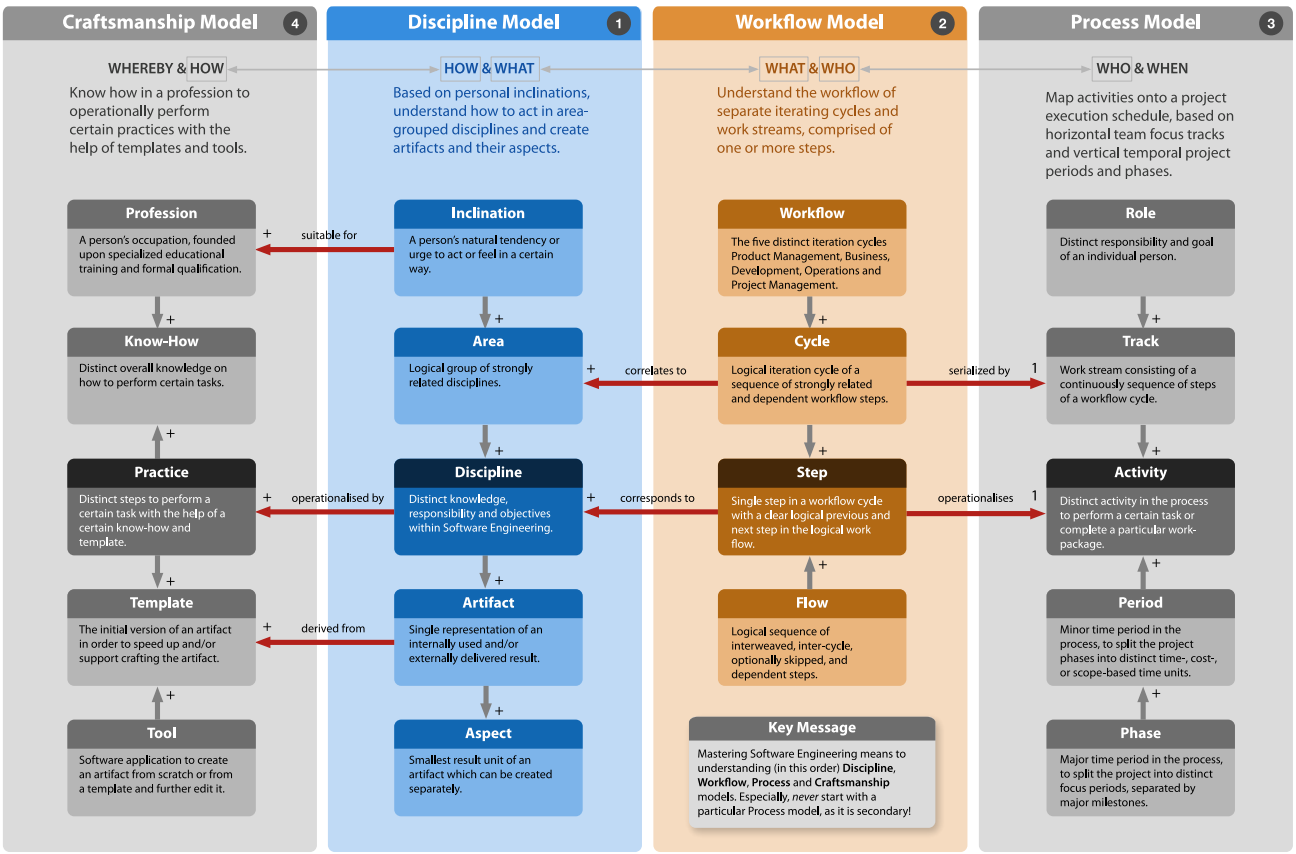
Software Engineering in Industrial Practice (SEIP)

Dr. Ralf S. Engelschall

ENGINEERING FUNDAMENTALS

Software Engineering Metamodel

TECHNISCHE UNIVERSITÄT MÜNCHEN



Intellectual content: Version 1.0.4 (2021-04-19), Authored 2021 by Dr. Ralf S. Engelschall (rse@engelschall.com)
Graphical illustration: Version 1.0.2 (2021-04-19), Copyright © 2021 Dr. Ralf S. Engelschall (http://engelschall.com) – All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

EF 02.1

big picture & coherency

Software Engineering can be understood through a meta-model based on four distinct but interlinked models.

The **Craftsmanship Model** is the base and targets the WHEREBY & HOW. It spans from the **Professions** of individual persons, their corresponding **Know-Hows** and **Practices** to the underlying **Templates** and **Tools**.

The **Discipline Model** targets the HOW & WHAT. It segregates Software Engineering into **Disciplines**, which are grouped into **Areas** and which are motivated by the usual **Inclinations** of individual persons. Each Discipline is then described through input and output **Artifacts** and their **Aspects**.

The **Workflow Model** targets the WHAT & WHO. It describes a **Workflow** of **Cycles** which contain **Steps**. A **Flow** is the run through those Steps over time.

The **Process Model** finally targets the WHO & WHEN. It maps **Activities** onto a project execution schedule, based on horizontal **Tracks** of **Roles** and vertical **Periods** of **Phases**.

Questions

- How many Cycles are known in the Workflow Model of Software Engineering, in which persons with similar Inclinations act?

ENGINEERING FUNDAMENTALS

Software Engineering Disciplines

TECHNISCHE UNIVERSITÄT MÜNCHEN



Intellectual Content: Version 3.0.0 (2021-12-10), Authored 2009-2022 by Dr. Ralf S. Engelschall and Rudolf Koster, Graphical Illustration: Version 3.3.2 (2021-12-04), Copyright © 2009-2021 Dr. Ralf S. Engelschall (http://engelschall.com), All Rights Reserved. Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science Lecture contexts only.

EF 02.2

inclination & knowledge

Software Engineering can be understood through 20 distinct **Disciplines** (operationalized through input and output artifacts and their aspects), which are logically grouped into 10 distinct **Areas**, and which in turn are logically grouped into 5 distinct **Inclinations** of individual persons.

Persons with a strong **domain-specific** and **business-oriented** Inclination act in the Areas **Analysis** and **Experience** and the corresponding Disciplines **Software Requirements**, **Domain Modeling**, **User Experience** and **User Interface**.

Persons with a strong **constructive** and **technological** Inclination act in the Areas **Architecture** and **Development** and the corresponding Disciplines **Software Architecture**, **System Architecture**, **Software Development** and **Software Refactoring**.

Persons with a strong **infrastructural** and **technological** Inclination act in the Areas **Configuration** and **Delivery** and the corresponding Disciplines **Software Versioning**, **Software Assembly**, **Software Deployment** and **Software Operations**.

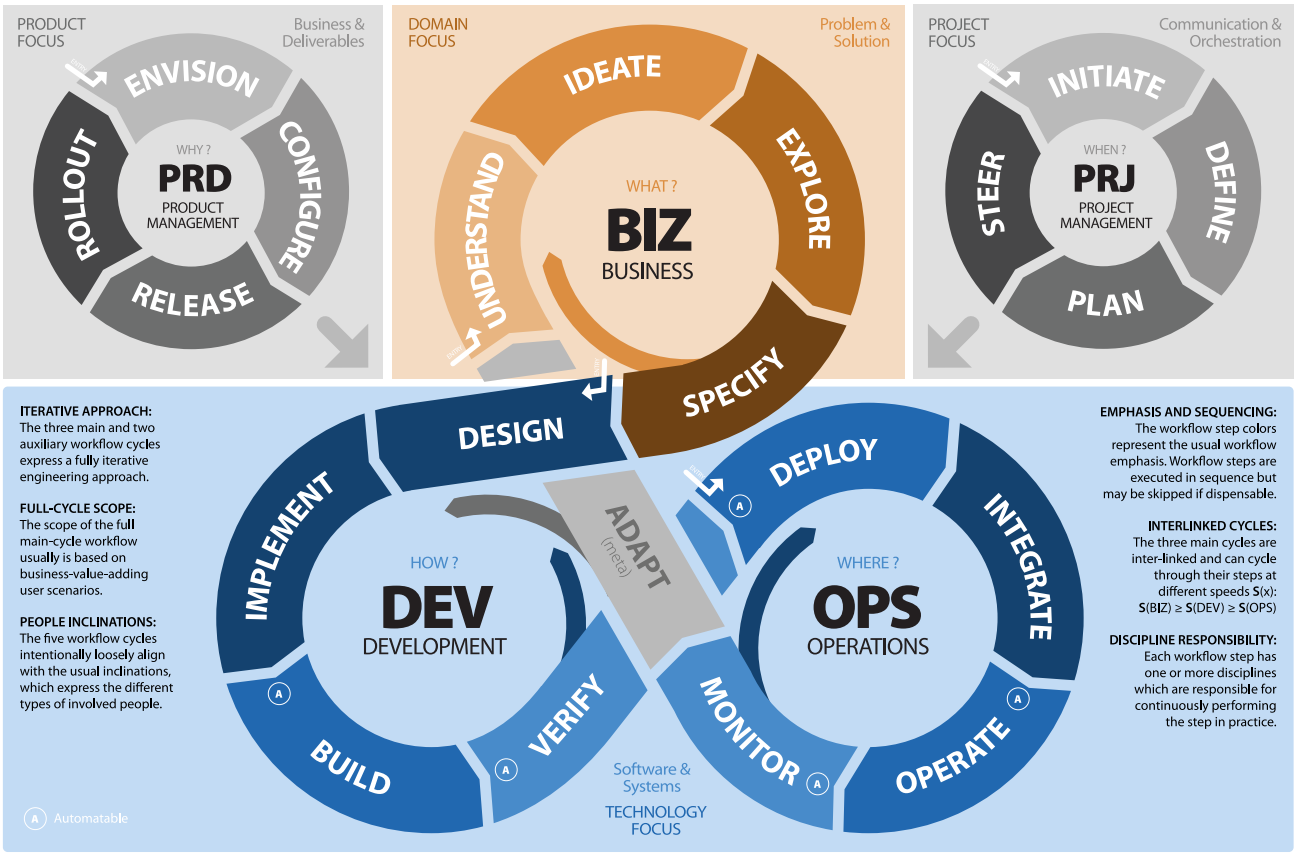
Persons with a strong **analytical** and **domain-specific** Inclination act in the Areas **Analytics** and **Comprehension** and the corresponding Disciplines **Software Reviewing**, **Software Testing**, **Usage Documentation** and **User Training**.

Persons with a strong **people-oriented** and **process-oriented** Inclination act in the Areas **Management** and **Adjustment** and the corresponding Disciplines **Project Management**, **Project Auditing**, **Project Coaching** and **Change Management**.

Questions

- Which Disciplines of Software Engineering are considered the **King Disciplines**?

Software Engineering Workflow



Intellectual Content: Version 1.1 (2021-12-29), Authored 2019-2021 by Dr. Ralf H. Engelschall
Graphics/Illustration: Version 1.1.0 (2021-12-29), Copyright © 2020-2021 Dr. Ralf H. Engelschall <http://engelschall.com>, All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture comments only.

The Workflow Model describes the work segregation in Software Engineering. In the **Workflow**, three main and two auxiliary workflow cycles express the iterative approach. The full main-cycle workflow is usually based on business-value-adding user scenarios.

The three main cycles are interlinked and can cycle through their steps at different speeds, S(x), with S(BIZ) greater then or equal to S(DEV), which in turn is greater then or equal to S(OPS). Because the cycles with earlier steps should not slow down the cycles with later steps.

The step colors represent workflow emphasis. Workflow steps are executed in sequence but may be skipped if dispensable.

The ten discipline areas of Software Engineering express the different roles of the involved people. The five inclinations express the different types of the involved people. Hence, the Workflow consists of exactly five cycles.

Questions

- What does the Workflow-Model of Software Engineering describe?

Software Engineering Steps



Intellectual Content: Version 1.0.1 (2021-04-13), Authored 2021 by Dr. Ralf S. Engelschall and Rudolf Koster.
Graphical Illustration: Version 1.0.1 (2021-04-13), Copyright © 2021 Dr. Ralf S. Engelschall <http://engelschall.com> – All Rights Reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

Software Engineering, on an operational level, can be alternatively understood through 20 distinct **Steps** which are continuously performed within the **Software Engineering Workflow**. Each Step belongs to one primarily responsible Discipline and zero or more secondarily responsible Disciplines.

Workflow Steps are the adequate concept to understand which activities have to be performed in each iteration of a **Software Engineering Process**.

Questions

- Which concept allows one to best understand which activities have be performed in a Software Engineering Process?

Software Engineering Process

1. WORKFLOW CYCLES
The workflow has five cycles which continuously iterate through their steps. Workflow steps are executed in each cycle in sequence, but may be skipped if dispensable in a particular iteration of the process. The length of an iteration is arbitrary, but can be e.g. about 1/3 of a Scrum sprint.

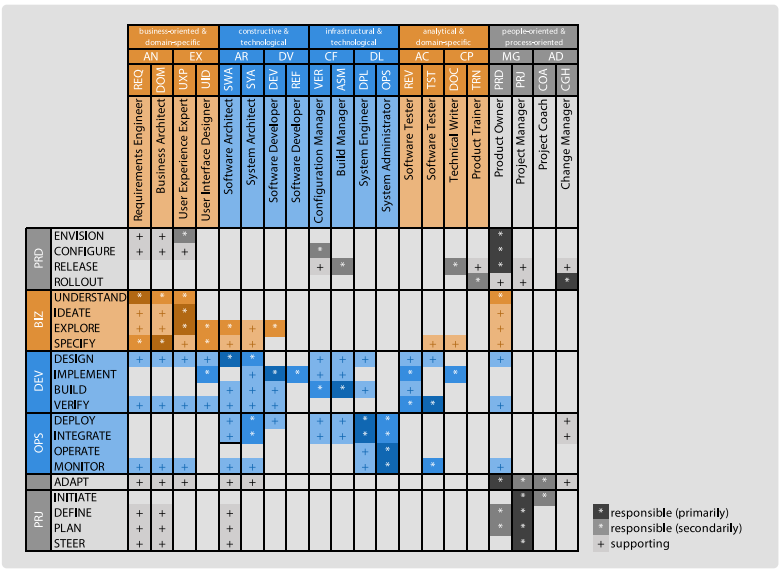
2. WORKFLOW STEPS:
The workflow steps describe a logical activity which has to be performed. Each step relates to one or more discipline areas and their corresponding disciplines, which express the operative responsibilities for each workflow step. In each discipline individual roles act.

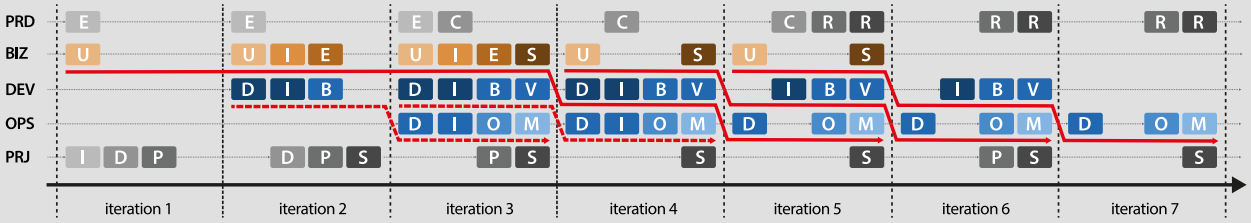
3. WORKFLOW ROLES:
The workflow roles are held by individual persons. Each role is primarily responsible for a particular workflow step. In addition, each role can be secondarily responsible for other workflow steps or at least actively support those steps.

4. PROJECT SCHEDULE:
To create a particular project execution schedule, the five cycles, their iterations and their steps have to be mapped onto a timeline. The cycles are mapped onto (horizontal) timeline tracks, the iterations are mapped onto (vertical) timeline phases, and the steps are mapped onto timeline activities.

5. PROCESS FLOWS (THE CRUX):
The activities across the cycles can (and should) be linked into individual (diagonal) waterfall-like flows, although the execution schedule, from the perspective of the cycles, is fully iterative. There are multiple such flows in parallel and they are usually highly interleaved on the project timeline in order to maximally utilize the team.

6. PROCESS ADAPTION:
In the meta-step ADAPT, the process is adapted by choosing which workflow steps are required for the next iteration. The major input for this decision is the current solution state and the feedback on it by the customer.





The workflow has five main cycles, which continuously iterate through their steps. Workflow steps are executed in each cycle in sequence but may be skipped if dispensable.

The workflow steps are annotated with discipline areas to express operative responsibilities. In each area, multiple roles act.

The workflow roles are held by individual persons. Each role is primarily responsible for a particular workflow step. In addition, each role can be secondarily responsible for other workflow steps or at least actively support those steps.

To create a particular project execution schedule, the five cycles, their iterations, and their steps have to be mapped onto a timeline. The cycles are mapped onto (horizontal) timeline tracks, and the iterations are mapped onto (vertical) timeline phases, and the steps are mapped onto timeline activities.

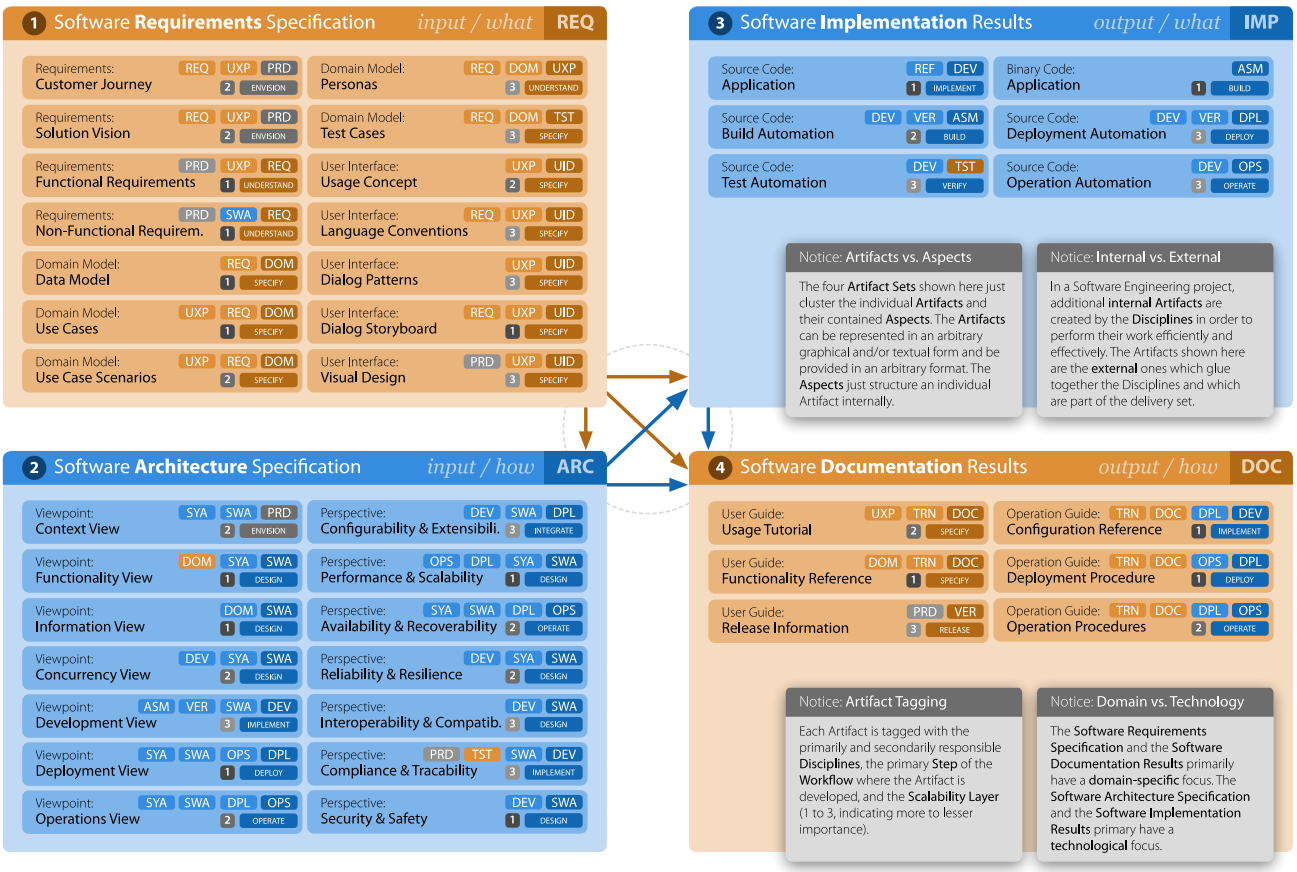
The activities across the cycles can (and should) be linked into individual (diagonal) waterfall-like flows, although the execution schedule, from the perspective of the cycles, is fully iterative. There are multiple such flows in parallel, and they are usually highly interleaved on the project timeline in order to maximally utilize the team.

Questions

- How can maximum utilization of the team be achieved in Software Engineering, despite a division of labor?

Intellectual Content: Version 1.0 (2021-12-18), Authored 2019-2021 by Dr. Ralf S. Engelschall
Graphic/Illustration: Version 1.1 (2021-12-18), Copyright © 2020-2021 Dr. Ralf S. Engelschall <http://engelschall.com>, All rights reserved.
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science for Business Students only.

Software Engineering Artifacts



Intellectual Content: Version 1.0.0 (2021-10-10), Authored 2021 by Dr. Ralf S. Engelschall
Graphical Illustration: Version 1.0.0 (2021-10-10), Copyright © 2021 Dr. Ralf S. Engelschall <http://engelschall.com> – All Rights Reserved
Unauthorized Reproduction Prohibited. Licensed to Technische Universität München (TUM) for reproduction in Computer Science lecture contexts only.

The four **Artifact Sets** just cluster the individual **Artifacts** and their contained **Aspects**. The **Artifacts** can be represented in an arbitrary graphical and/or textual form and be provided in an arbitrary format. The **Aspects** just structure an individual Artifact internally.

In a Software Engineering project, additional **internal** Artifacts are created by the **Disciplines** in order to perform their work efficiently and effectively. The shown Artifacts are just the **external** ones which glue together the Disciplines and which are part of the delivery set.

Each **Artifact** is tagged with the primary and secondary responsible **Disciplines**, the primary **Step** of the Workflow where the Artifact is developed, and the Scalability Layer (1 to 3, indicating more to lesser importance).

The **Software Requirements Specification** and the **Software Documentation Results** primarily have a **domain-specific** focus. The **Software Architecture Specification** and the **Software Implementation Results** primarily have a **technological** focus.

Questions

- What focus has the **Software Requirements Specification**?